Laboratory Experiment 02

Reinforcement-Learning-Based Agents versus Rule-Based Agents

Intelligent System Development

Academic Year 2026–2027

Submission deadline: 11:30 PM, Thursday, 29 October 2026.
Required file name: `agent02_class_team_name.pdf`
Submission format: PDF document.
Working arrangement: Follow the instructor's instructions regarding individual or team work.
Required evidence: Python source code, experimental results, tables or figures, and written answers to all 10 questions.

Contents

1 Objectives **3**

2 Problem Description **3**

2.1 Agent A: Reinforcement-learning-based agent . . . 3

2.2 Agent B: Rule-based agent . . . 4

3 Environment and Mathematical Model **4**

3.1 State space . . . 4

3.2 Action space . . . 4

3.3 Customer conversion . . . 4

3.4 Sales and profit . . . 5

3.5 Environment dynamics . . . 5

4 Method 1: Q-Learning Agent **5**

4.1 Action-value function . . . 5

4.2 Q-learning update . . . 6

4.3 Exploration and exploitation . . . 6

4.4 Algorithm . . . 6

5 Method 2: Rule-Based Agent **7**

6 Implementation Requirements **7**

6.1 Reference parameters . . . 8

7 Experimental Design **8**
7.1 Experiment A: Run the baseline 8
7.2 Experiment B: Examine learning 8
7.3 Experiment C: Change the learning rate 9
7.4 Experiment D: Change exploration 9
7.5 Experiment E: Compare policies fairly 9
7.6 Suggested results table 9

8 Ten Laboratory Questions **10**

9 Discussion and Interpretation **11**

10 Submission Requirements **11**
10.1 File naming and deadline 12

11 Assessment Rubric **12**

12 Conclusion **12**

1 Objectives

The purpose of this laboratory experiment is to understand, implement, and experimentally compare two approaches to intelligent agent design:

1. Reinforcement-learning-based agents.
2. Rule-based agents.

After completing this experiment, students should be able to:

1. Explain the concepts of an agent, environment, state, action, reward, and policy.
2. Implement a simple reinforcement-learning agent using tabular Q-learning.
3. Implement a rule-based agent using conditional statements.
4. Design a small multi-agent simulation.
5. Collect, visualize, and interpret experimental data.
6. Compare agents using consistent performance metrics.
7. Explain the limitations of learning-based and rule-based decision-making.

2 Problem Description

Consider a simplified e-commerce marketplace in which two agents sell similar products. Each agent must decide whether to offer a discount to customers.

The agents interact with a shared environment. Customer demand may be low or high, and each agent can observe whether its competitor offered a discount in the previous time step.

The available actions are:

- Action 0: Do not offer a discount.
- Action 1: Offer a discount.

The objective is to obtain a high cumulative profit while responding to changing demand and competitor behavior.

We investigate two different ways to make decisions.

2.1 Agent A: Reinforcement-learning-based agent

The reinforcement-learning (RL) agent learns from experience. It observes a state, selects an action, receives a reward, and updates its estimate of the value of that action.

In this experiment, the agent uses tabular Q-learning.

2.2 Agent B: Rule-based agent

The rule-based agent follows a predefined decision policy. For example, it may avoid discounts when demand is low and match a competitor's discount when demand is high.

Its behavior is determined by manually specified rules rather than a learning algorithm.

Important distinction: A rule-based agent does not automatically learn from rewards. An RL-based agent uses experience and rewards to update its action-value estimates.

3 Environment and Mathematical Model

3.1 State space

For agent $i \in \{1, 2\}$, define the local state at time t as

$$s_i(t) = (d_t, c_i(t)),$$

where:

- $d_t \in \{0, 1\}$ represents market demand: 0 for low demand and 1 for high demand.
- $c_i(t) \in \{0, 1\}$ represents the competitor's previous action: 0 for no discount and 1 for discount.

The environment therefore provides four possible states to each agent.

The state is a simplified representation of the market. It does not contain all information about customer preferences, competitor strategies, or future demand.

3.2 Action space

Each agent selects an action

$$a_i(t) \in \mathcal{A} = \{0, 1\}.$$

The two agents act simultaneously. Consequently, agent i must make its decision without observing the other agent's current action.

3.3 Customer conversion

Let $p_i(t)$ be the probability that a potential customer purchases from agent i. In the reference implementation,

$$p_i(t) = \text{clip}\left(0.30 + 0.25d_t + 0.15a_i(t) - 0.10a_{-i}(t),\, 0.05,\, 0.95\right),$$

where $a_{-i}(t)$ denotes the competitor's current action.

The function clip restricts the probability to the interval $[0.05, 0.95]$.

This formula is an illustrative model rather than a validated economic law. It assumes that discounts can increase conversion and that competitor discounts can reduce an agent's conversion.

3.4 Sales and profit

The number of potential customers is

$$N_t = \begin{cases} 10, & d_t = 0, \\ 20, & d_t = 1. \end{cases}$$

The number of sales is sampled from a binomial distribution:

$$S_i(t) \sim \text{Binomial}(N_t, p_i(t)).$$

Let the original product price be $P = 10$ and the unit cost be $C = 5$. The discount rate is $\delta = 0.20$ when the agent offers a discount and zero otherwise.

The effective price is

$$P_i(t) = P\big(1 - \delta a_i(t)\big).$$

The reward, interpreted as profit, is

$$r_i(t) = S_i(t)\big(P_i(t) - C\big).$$

Thus, a discount can increase the probability of making sales but reduces the profit margin on each sale.

The agent must balance these effects.

3.5 Environment dynamics

After both agents receive their rewards, the environment generates the next demand state independently of the current actions. The competitor's previous action becomes the competitor's current action.

The reference model is intentionally simple. In particular, the demand process is independent of the agents' decisions, and the current conversion model is illustrative.

4 Method 1: Q-Learning Agent

4.1 Action-value function

The RL agent maintains a Q-table:

$$Q : \mathcal{S} \times \mathcal{A} \longrightarrow \mathbb{R}.$$

For each state-action pair, $Q(s, a)$ estimates the expected discounted return obtained by taking action a in state s and subsequently following the learned policy.

The agent has four states and two actions. Therefore, its Q-table contains eight action-value entries.

4.2 Q-learning update

After observing the reward and next state, the agent updates the selected Q-value according to

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a \in \mathcal{A}} Q(s_{t+1}, a) - Q(s_t, a_t) \right],$$

where:

- α is the learning rate.
- γ is the discount factor.
- r_t is the reward received at time t.
- s_{t+1} is the next observed state.

The learning rate controls how strongly new information changes the existing Q-value. The discount factor controls the importance of estimated future rewards.

4.3 Exploration and exploitation

The reference agent uses an ε-greedy strategy:

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \varepsilon, \\ \arg\max_a Q(s_t, a), & \text{otherwise.} \end{cases}$$

Exploration allows the agent to try actions whose values are not yet well understood. Exploitation selects the action with the highest estimated value.

The reference implementation gradually decreases ε during training, while maintaining a small minimum exploration probability.

4.4 Algorithm

1. Initialize the Q-table to zero.
2. Reset the environment and observe each agent's state.
3. Select an action using the ε-greedy policy.
4. Execute both agents' actions simultaneously.
5. Observe rewards and next states.
6. Update each agent's Q-table.
7. Repeat for the specified number of steps.
8. Evaluate the learned policies after training.

5 Method 2: Rule-Based Agent

The rule-based agent does not maintain a Q-table. Its action is determined directly by the observed state.

The reference policy is

$$\pi^{\text{rule}}(d, c) = \begin{cases} 0, & d = 0, \\ 1, & d = 1 \text{ and } c = 1, \\ 0, & d = 1 \text{ and } c = 0. \end{cases}$$

The corresponding rules are:

1. If demand is low, do not offer a discount.
2. If demand is high and the competitor previously offered a discount, offer a discount.
3. If demand is high and the competitor did not previously offer a discount, do not offer a discount.

These rules are manually designed. They do not necessarily maximize profit.

A rule-based system may perform well when its rules match the environment. However, when conditions change, the rules may need to be redesigned by a developer.

6 Implementation Requirements

Students should use the provided Python implementation as a starting point. The code uses NumPy and does not require a GPU.

Install the dependency if necessary:

```
pip install numpy matplotlib
```

Save the program as:

`agent02_experiment.py`

The program should contain the following components:

1. An environment class that implements state initialization, actions, rewards, and state transitions.
2. A Q-learning agent class.
3. A rule-based agent class.
4. Separate functions for training and evaluating the two approaches.
5. A main program that executes the experiments and records the results.

Students may modify the reference implementation, provided that they document their changes and preserve a valid, reproducible comparison.

6.1 Reference parameters

Parameter	Reference value
Number of agents	2
Episodes	5,000
Steps per episode	20
Learning rate α	0.10
Discount factor γ	0.90
Initial exploration ε	0.20
Minimum exploration	0.02
Discount rate	20%
Original price	10
Unit cost	5
Random seed	42 for the environment

The reference code also uses separate random-number generators for the two RL agents. Keep these seeds fixed when reproducing the baseline results.

7 Experimental Design

7.1 Experiment A: Run the baseline

Run both agent types using the same environment model.

Record the following quantities:

- Total reward for each agent.
- Average reward per time step.
- Number of discount and no-discount actions.
- Learned Q-values and policy for each RL agent.

For a run of T time steps, define the average reward of agent i as

$$\overline{R}_i = \frac{1}{T} \sum_{t=1}^{T} r_i(t).$$

A higher average reward indicates higher average profit under the chosen simulation model.

7.2 Experiment B: Examine learning

Record the cumulative reward during training. Plot the average reward over consecutive windows of episodes.

For a window containing K episodes, the mean reward per time step can be calculated as

$$\overline{R}_{i,k} = \frac{\sum_{t \in \mathcal{W}_k} r_i(t)}{20K},$$

where $\mathcal{W}_k$ denotes the time steps in the window.

Explain whether performance improves, remains stable, or fluctuates. Do not assume that a rising training reward automatically proves convergence or good generalization.

7.3 Experiment C: Change the learning rate

Run at least three configurations:

$$\alpha \in \{0.01, 0.10, 0.50\}.$$

Keep the other parameters unchanged.

Compare the reward curves and final evaluation rewards. Discuss whether faster updating improves learning stability or causes excessive fluctuations.

7.4 Experiment D: Change exploration

Run at least three configurations with different exploration settings. For example, compare the reference schedule with a fixed $\varepsilon = 0.05$ and a fixed $\varepsilon = 0.30$.

Record both training and evaluation performance. Explain the trade-off between exploring actions and exploiting learned action values.

7.5 Experiment E: Compare policies fairly

Evaluate the trained RL agents and rule-based agents in separate evaluation runs. Use the same demand-generation procedure and comparable random seeds.

For evaluation, exploration should be disabled or fixed to a clearly documented small value. For a greedy evaluation, set $\varepsilon = 0$.

The original training loop does not provide a fully isolated evaluation phase. Students should implement this separation before drawing strong conclusions about policy quality.

For additional reliability, repeat the experiments with multiple independent seeds and report the mean and standard deviation of average reward.

7.6 Suggested results table

Agent/configuration	Mean reward	Std. dev.	Discount rate
RL, baseline			
RL, low learning rate			
RL, high learning rate			
RL, alternative exploration			
Rule-based			

Use actual measured results. Do not invent or copy expected values in place of experimental evidence.

8 Ten Laboratory Questions

Answer all ten questions. Support your answers with relevant code, calculations, tables, figures, or observations from your experiments.

Question 1. Agent and environment modeling

Identify the agents, environment, state space, action space, reward function, and state-transition process in the e-commerce simulation.

Explain why this problem can be modeled as a multi-agent system rather than a single-agent system.

Question 2. Learning-based versus rule-based decisions

Compare the RL-based agent and the rule-based agent in terms of decision-making, use of experience, adaptability, and computational requirements.

Give one practical situation in which a rule-based agent could be preferable and another in which an RL-based agent could be preferable.

Question 3. Q-learning update

Explain the mathematical meaning of the Q-learning update equation. Using a hypothetical example, calculate the updated Q-value when

$$Q(s, a) = 4, \quad \alpha = 0.1, \quad r = 6, \quad \gamma = 0.9,$$

and

$$\max_{a'} Q(s', a') = 5.$$

Show each calculation step and interpret the result.

Question 4. Exploration and exploitation

Explain the role of ε-greedy action selection. Compare at least two exploration configurations using your experimental results.

What may happen if the agent explores too much or stops exploring too early?

Question 5. Reward and business objectives

Explain why offering a discount may increase sales but decrease profit per sale.

Using the specified price, unit cost, and discount rate, calculate the profit per sale with and without a discount.

Discuss how changing the reward function could alter the agent's learned behavior.

Question 6. Performance evaluation

Calculate the average reward per time step for each agent from your experimental data.

Compare their performance using a table or figure. Explain why one training run is insufficient to establish that one approach is generally superior.

Question 7. Learning rate and stability

Compare the Q-learning results for at least three learning rates.

Describe how the learning rate affects updating, stability, and sensitivity to new observations. Support your discussion with measured results.

Question 8. Multi-agent interaction and non-stationarity

Explain how the other agent's actions influence an agent's rewards and observations.

Why can independent Q-learning become difficult when both agents learn simultaneously? Suggest one experimental change that could help investigate this problem.

Question 9. Limitations and robustness

Identify at least three limitations of the reference simulation or its assumptions.

Propose and implement one modification, such as changing demand dynamics, discount size, customer conversion, or competitor behavior. Explain how the modification affects the results.

Question 10. Design a better intelligent agent

Propose an improved agent or experimental framework for the e-commerce problem.

Specify its state representation, actions, reward function, learning or decision-making method, and evaluation procedure.

Explain how you would test whether your proposed approach is more robust than both the reference RL agent and the reference rule-based agent.

9 Discussion and Interpretation

In addition to answering the questions, the report should discuss the following general observations where relevant:

- A rule-based agent can make consistent decisions, but its performance depends on the quality of its rules.
- An RL-based agent can improve its decisions through feedback, but learning does not guarantee an optimal policy.
- Random demand and stochastic sales can cause reward variation even when the policy remains unchanged.
- Two simultaneously learning agents can influence each other's observed outcomes.
- Training performance and evaluation performance measure different aspects of an agent.

Students should distinguish observations from explanations. For example, a higher measured reward is an observation; the claim that a particular learning mechanism caused the improvement requires further evidence.

10 Submission Requirements

Submit one PDF report containing the following sections:

1. Title page with student or team identification.

2. Introduction and objectives.
3. Description of the environment and both agents.
4. Experimental setup and parameter configurations.
5. Results, tables, and figures.
6. Answers to all ten laboratory questions.
7. Discussion, limitations, and proposed improvements.
8. Conclusion.
9. References, if external materials are used.

Include the essential Python code or selected code extracts in the report. Full source files may be retained separately according to the instructor's submission instructions.

All results must be reproducible to a reasonable extent. State the random seeds, parameter values, number of episodes, and evaluation protocol.

10.1 File naming and deadline

The submitted PDF must use the following naming convention:

`agent02_class_team_name.pdf`

Replace `class` and `team_name` with the appropriate class identifier and team name.

Deadline: 11:30 PM, Thursday, 29 October 2026.

Students are responsible for checking that the PDF opens correctly, contains all required sections, and is submitted before the deadline.

11 Assessment Rubric

Criterion	Weight
Understanding of agents and mathematical models	15%
Correct implementation and reproducibility	20%
Experimental design and performance measurement	20%
Analysis and answers to the ten questions	30%
Discussion, limitations, and proposed improvements	10%
Report clarity and organization	5%
Total	**100%**

12 Conclusion

This laboratory introduces two fundamental approaches to intelligent decision-making. The RL-based agent learns action values from reward feedback, whereas the rule-based agent follows an explicitly designed policy.

Through implementation and controlled experiments, students should develop a practical understanding of learning, decision rules, stochastic environments, reward optimization, and interactions among agents.

The central objective is not merely to determine which agent obtains the highest reward in one simulation. It is to explain why the observed outcomes occur, evaluate their reliability, identify the assumptions behind them, and propose better methods for intelligent system design.